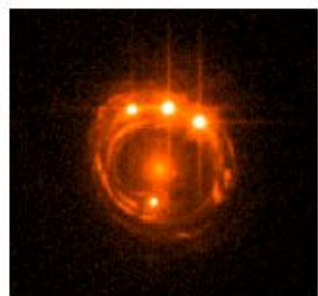


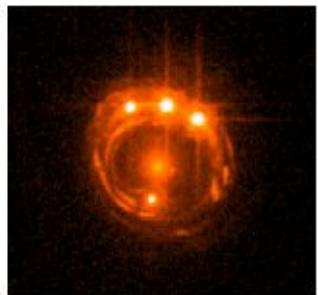
Case Study: Black Holes, JAX and GPUs

James Nightingale

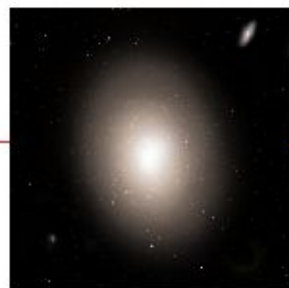
Contact: James.Nightingale@newcastle.ac.uk



$z = 0.5 - 2.0$



Source

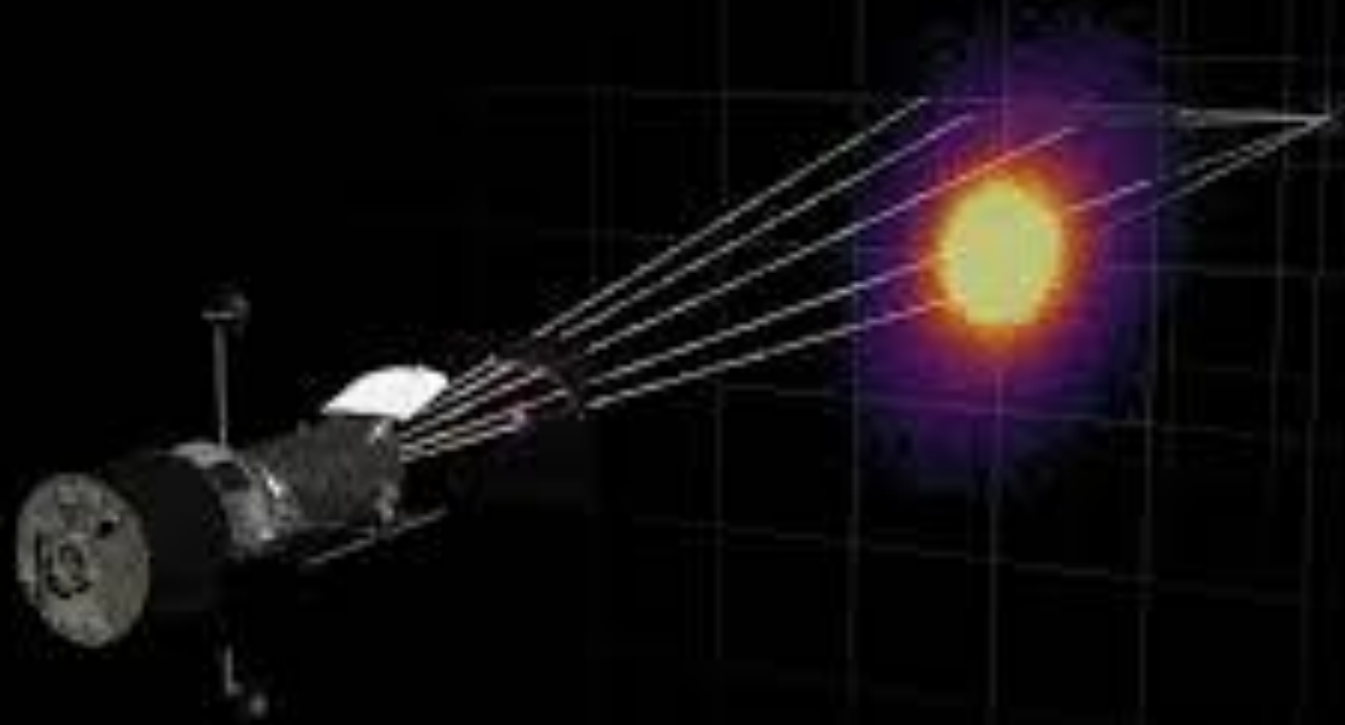


Lens

Observer



$z = 0.2 - 0.8$



Link to Youtube video on previous slide:

https://www.youtube.com/watch?v=2iUcpPn9ECg&embeds_referring_euri=https%3A%2F%2Fdocs.google.com%2F&embeds_referring_origin=https%3A%2F%2Fdocs.google.com&source_ve_path=Mjg2NjY

PyAutoLens: Open Source Lensing

GitHub:

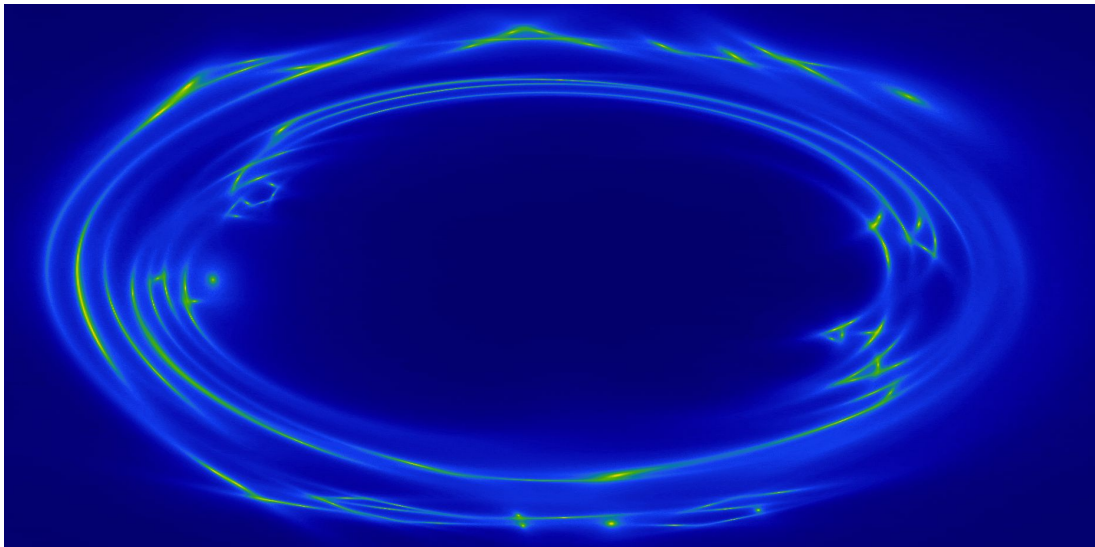
<https://github.com/Jammy2211/PyAutoLens>

Readthedocs:

<https://pyautolens.readthedocs.io/en/latest/>

JOSS paper:

<https://github.com/Jammy2211/PyAutoLens/blob/master/paper/paper.md>

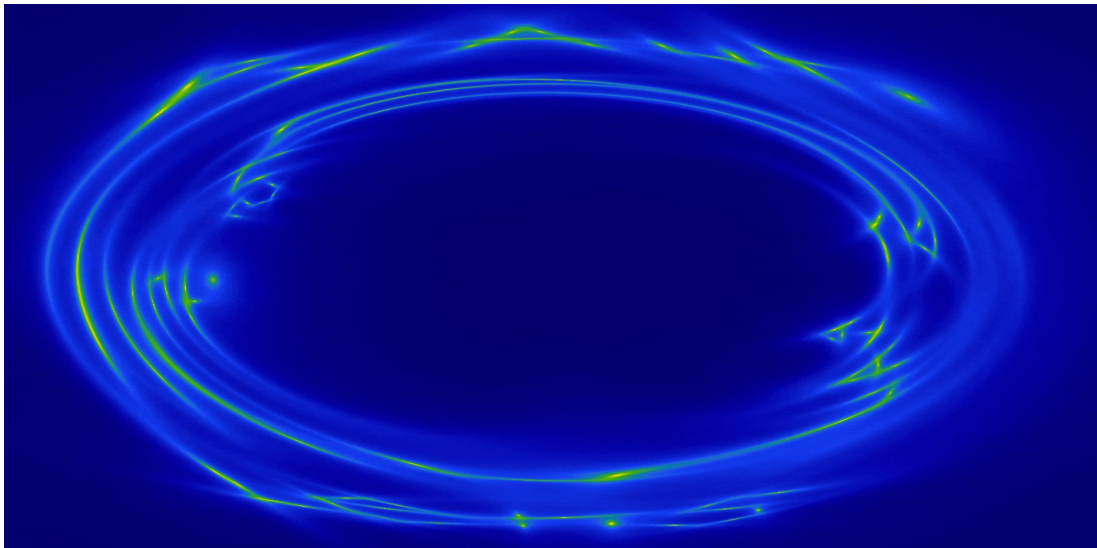


PyAutoLens: Open Source Lensing

Likelihood Function:

- Ray tracing (e.g. numpy arithmetic).
- 2D Convolution (real space or FFT).
- Linear Algebra & Matrix Calculations.
- Spatial geometry and meshes (e.g. Delaunay Voronoi).

**All performed in model-fitting
/ sampling (e.g. MCMC
analysis).**



JAX: A Powerful Python Library For GPUs

Developed by Google, 3 take home points:

- 1) **Useable:** Mirrors NumPy API such that a code-enthusiastic PhD student can have success using it (unlike CUDA).
- 2) **Extensive ecosystem:** many open-source libraries extending its functionality meaning it may be very amenable to your problem/
- 3) **Fast:** If you get your code fully JAX-ed, the speed up is significant, turn PyAutoLens analyses from 1 day on 20+CPU to < 30 minutes on a GPU.



JAX: The Downside

Likelihood Function:

- Ray tracing (e.g. numpy arithmetic).
- 2D Convolution (real space or FFT).
- Linear Algebra & Matrix Calculations.
- Spatial geometry and meshes (e.g. Delaunay Voronoi).

All performed in model-fitting / sampling (e.g. MCMC analysis).

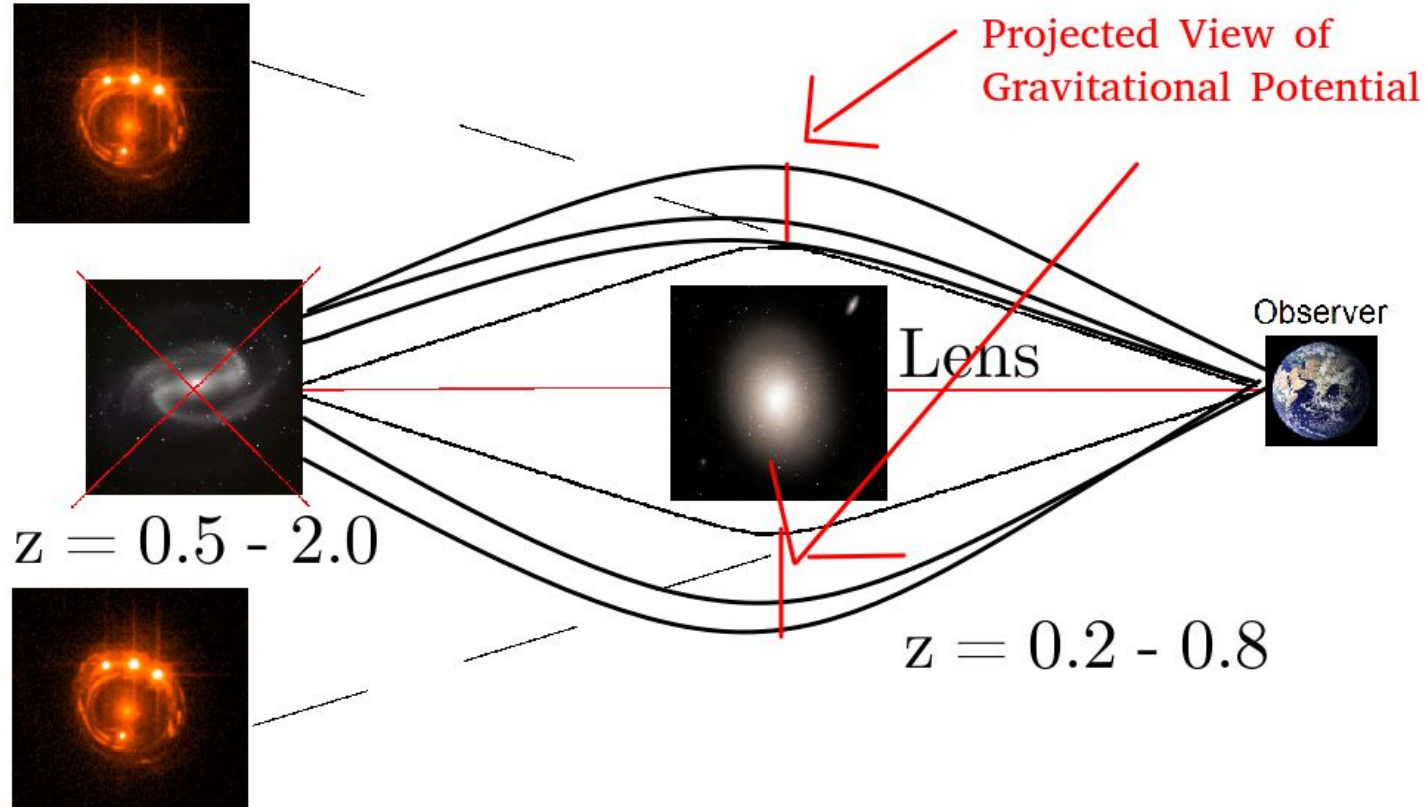
If one step in your calculation cannot be easily converted to JAX, this pretty much ruins the whole endeavor.

I spent 10% of my time converting 90% of PyAutoLens to JAX, and then the remaining 90% of time converting the remaining 10%.

Will you hit a “brick wall” converting your code to JAX?

Example 1: Trivial Conversion from NumPy

Example 1: Ray Tracing



Ray Tracing Example: A simple GPU conversion

$$\vec{\beta} = \vec{\theta} - \theta_E \frac{\vec{\theta}}{|\vec{\theta}|}$$

- $\vec{\theta} = (\text{image_x}, \text{image_y})$ is the **image plane coordinate**
- $\vec{\beta} = (\text{source_x}, \text{source_y})$ is the **source plane coordinate**
- θ_E (Einstein radius): Characteristic mass of the lens.

```
import numpy as np
```

```
r = np.sqrt(np.square(image_plane_x) + np.square(image_plane_y))
```

```
source_plane_x = np.subtract(image_plane_x, np.multiply(theta_E, np.divide(image_plane_x, r)))
```

```
source_plane_y = np.subtract(image_plane_y, np.multiply(theta_E, np.divide(image_plane_y, r)))
```

Ray Tracing Example: A simple GPU conversion

$$\vec{\beta} = \vec{\theta} - \theta_E \frac{\vec{\theta}}{|\vec{\theta}|}$$

- $\vec{\theta} = (\text{image_x}, \text{image_y})$ is the **image plane coordinate**
- $\vec{\beta} = (\text{source_x}, \text{source_y})$ is the **source plane coordinate**
- θ_E (Einstein radius): Characteristic mass of the lens.

```
import numpy as np
```

```
r = np.sqrt(np.square(image_plane_x) + np.square(image_plane_y))
```

```
source_plane_x = np.subtract(image_plane_x, np.multiply(theta_E, np.divide(image_plane_x, r)))
```

```
source_plane_y = np.subtract(image_plane_y, np.multiply(theta_E, np.divide(image_plane_y, r)))
```

Ray Tracing Example: A simple GPU conversion

$$\vec{\beta} = \vec{\theta} - \theta_E \frac{\vec{\theta}}{|\vec{\theta}|}$$

- $\vec{\theta} = (\text{image_x}, \text{image_y})$ is the **image plane coordinate**
- $\vec{\beta} = (\text{source_x}, \text{source_y})$ is the **source plane coordinate**
- θ_E (Einstein radius): Characteristic mass of the lens.

```
import jax.numpy as jnp
```

```
r = jnp.sqrt(jnp.square(image_plane_x) + jnp.square(image_plane_y))
```

```
source_plane_x = jnp.subtract(image_plane_x, jnp.multiply(theta_E, jnp.divide(image_plane_x, r)))
```

```
source_plane_y = jnp.subtract(image_plane_y, jnp.multiply(theta_E, jnp.divide(image_plane_y, r)))
```

Ray Tracing Example: A simple GPU conversion

$$\vec{\beta} = \vec{\theta} - \theta_E \frac{\vec{\theta}}{|\vec{\theta}|}$$

- $\vec{\theta} = (\text{image_x}, \text{image_y})$ is the **image plane coordinate**
- $\vec{\beta} = (\text{source_x}, \text{source_y})$ is the **source plane coordinate**
- θ_E (Einstein radius): Characteristic mass of the lens.

```
import jax.numpy as jnp
```

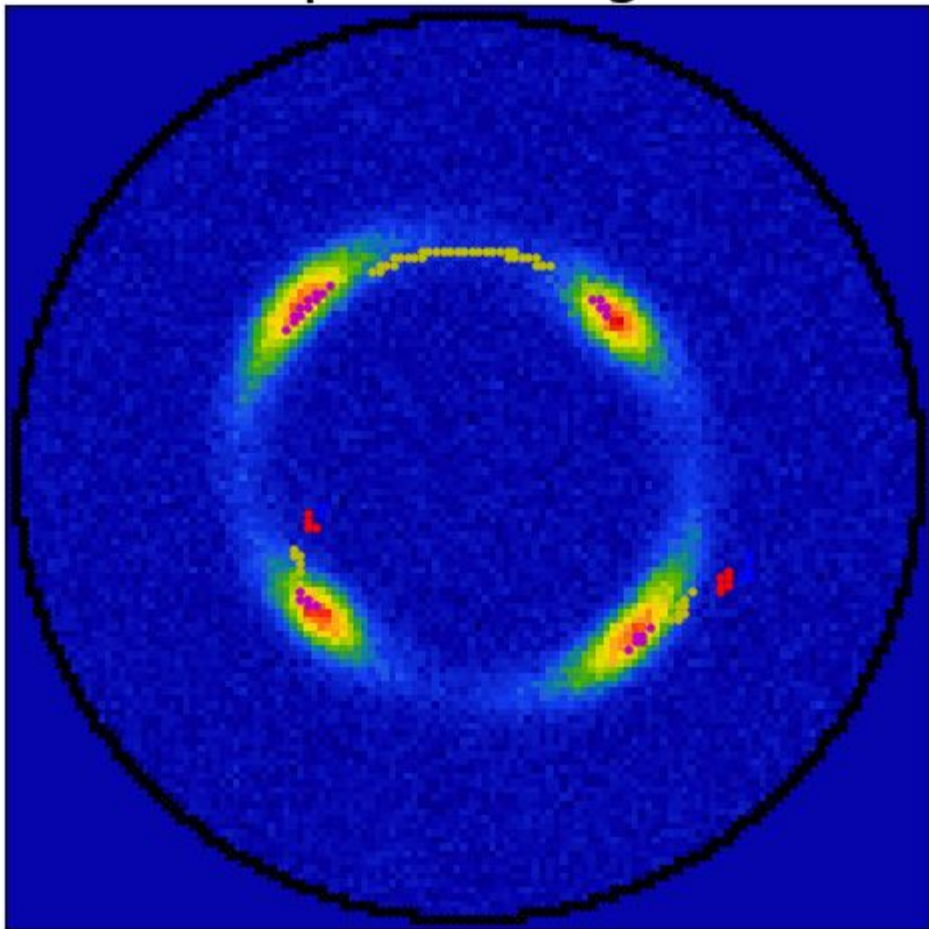
```
r = jnp.sqrt(jnp.square(image_plane_x) + jnp.square(image_plane_y))
```

```
source_plane_x = jnp.subtract(image_plane_x, jnp.multiply(theta_E, jnp.divide(image_plane_x, r)))
```

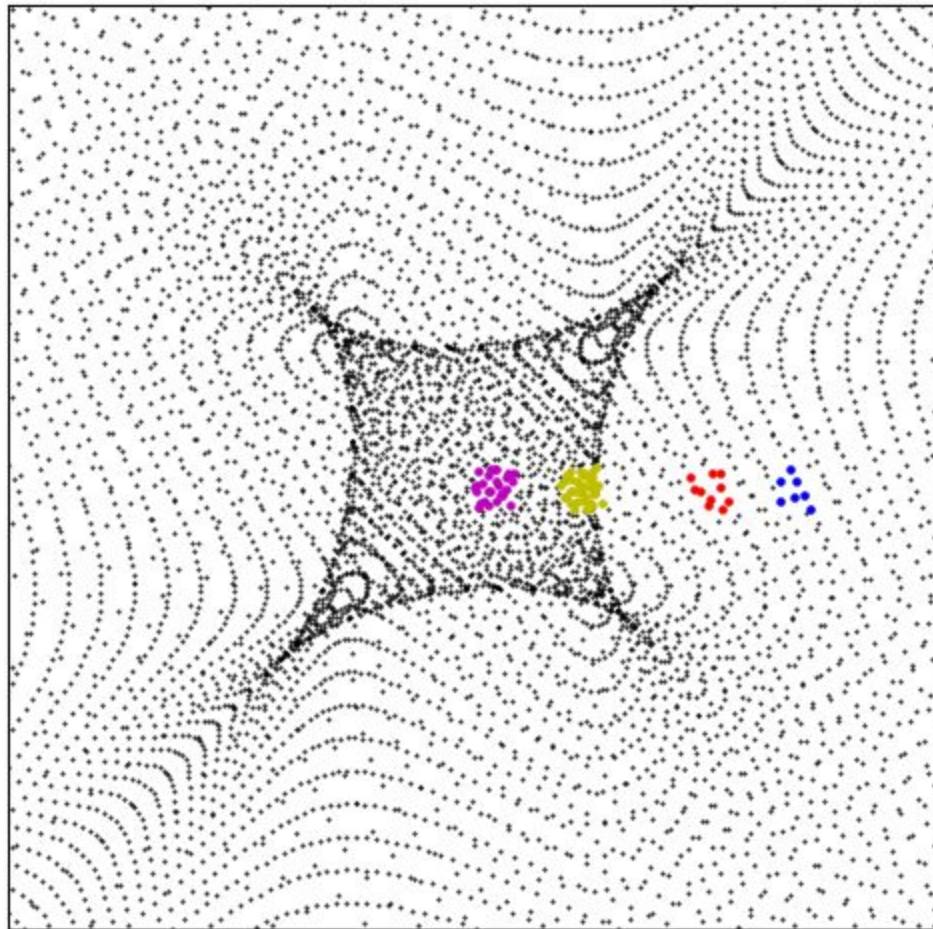
```
source_plane_y = jnp.subtract(image_plane_y, jnp.multiply(theta_E, jnp.divide(image_plane_y, r)))
```

Example 2: Writing as Vectorized (Numpy) Code

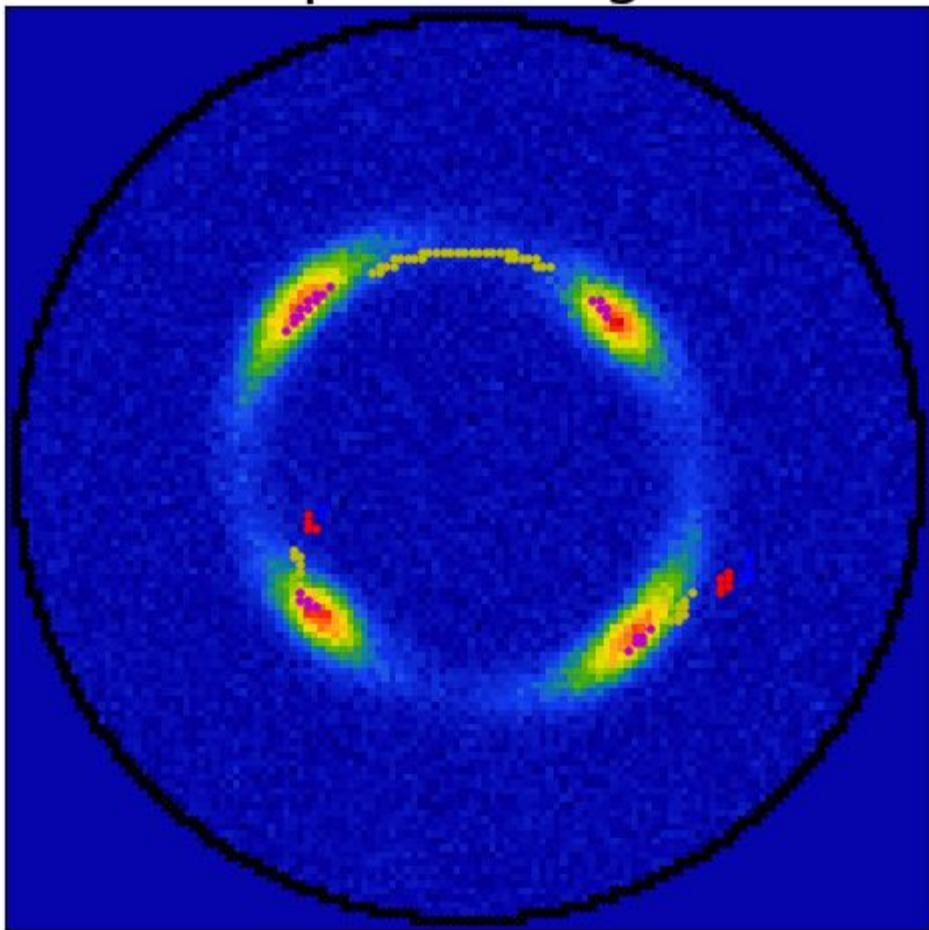
Example Strong Lens



Source Plane Grid



Example Strong Lens



Source Reconstruction

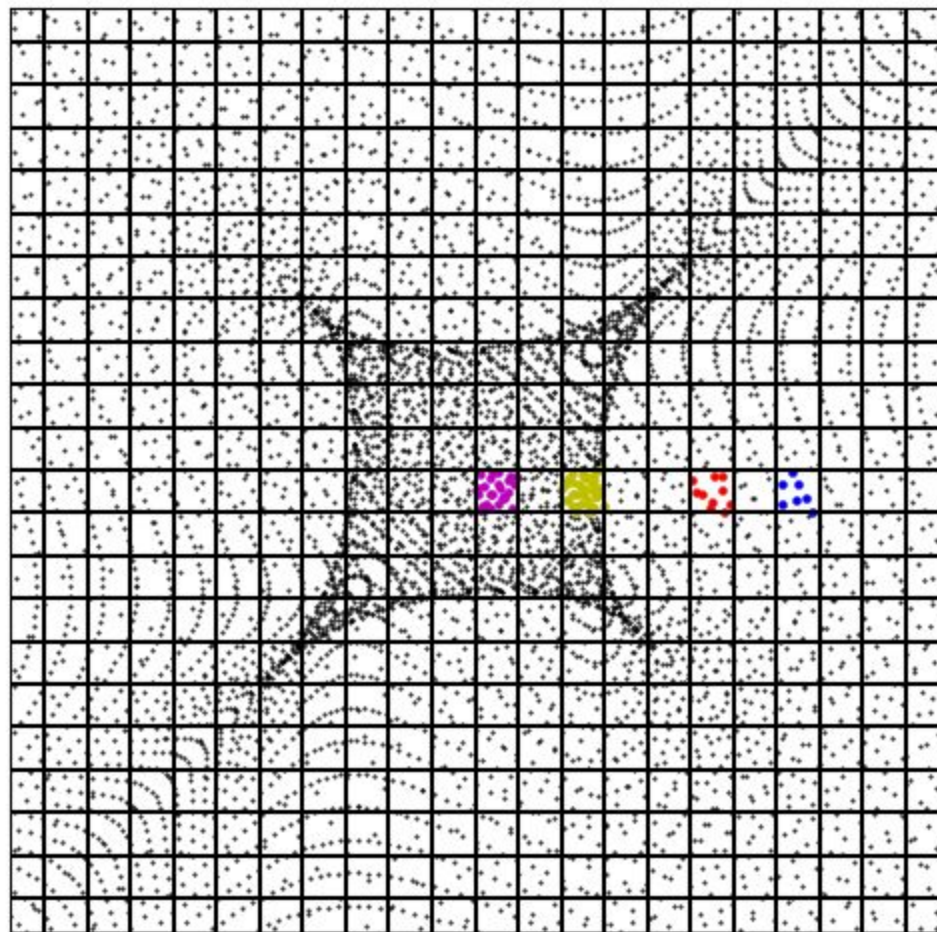
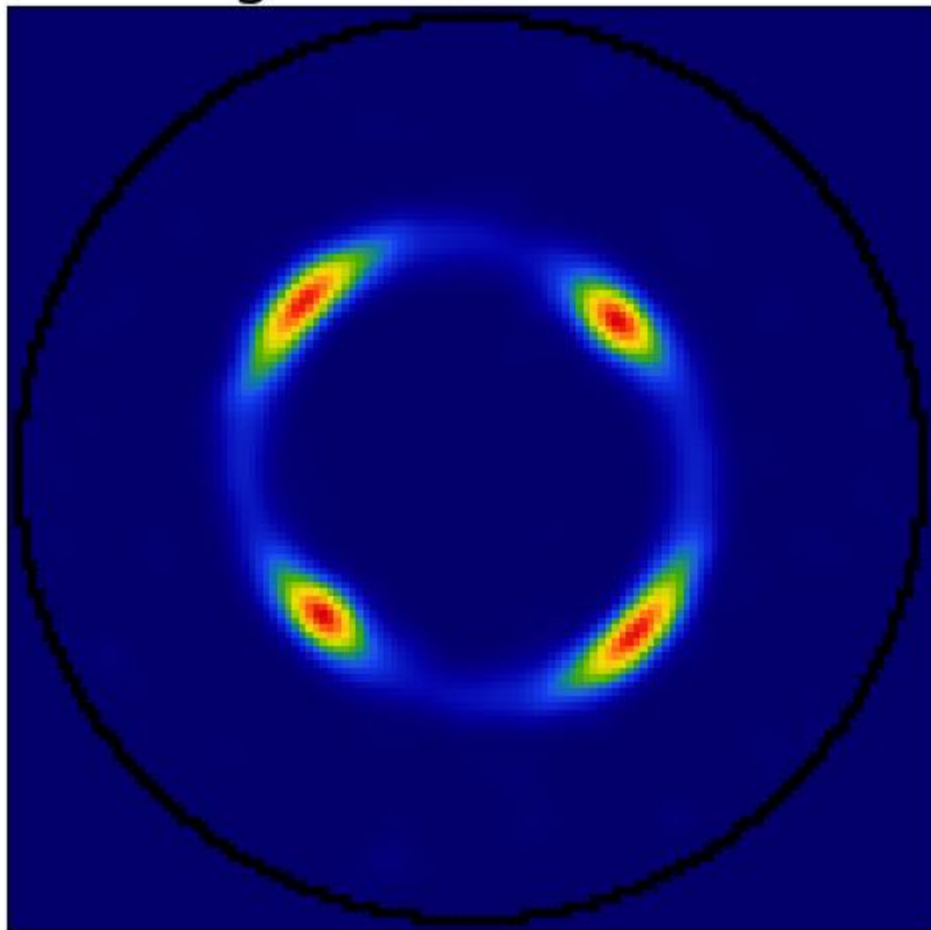
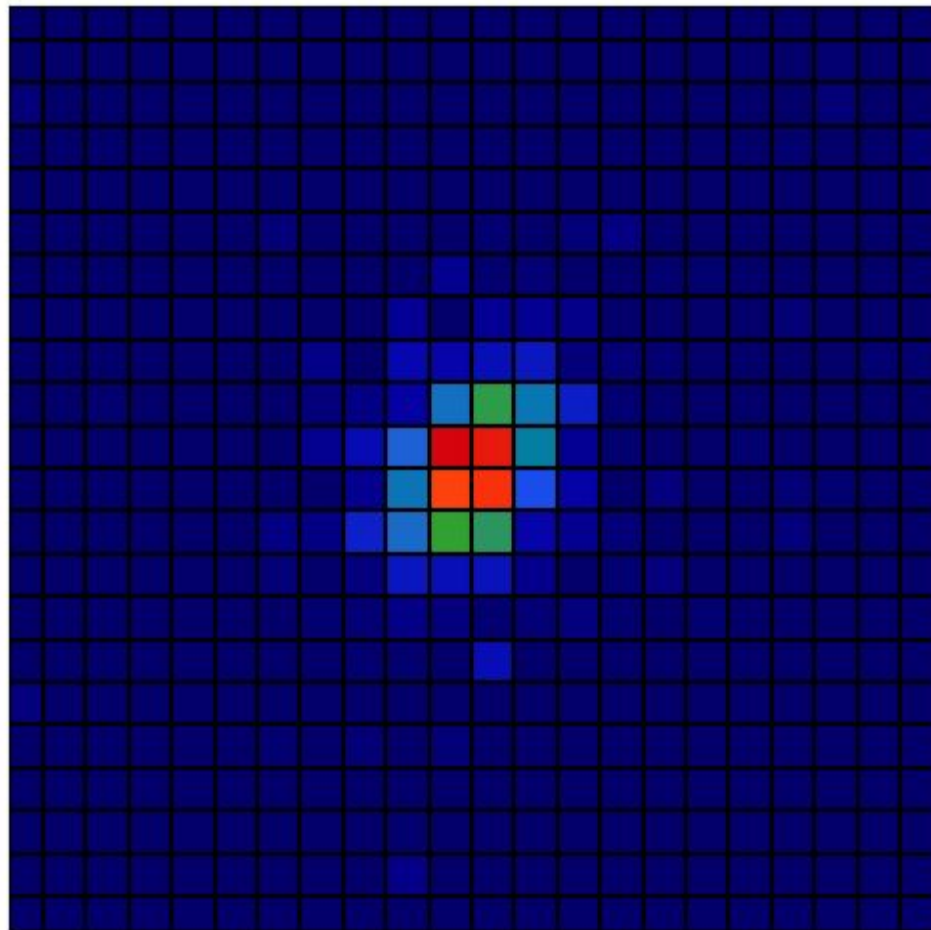


Image Reconstruction



Source Reconstruction



Mapping Matrix

We now define a key quantity in lens modeling called the **mapping matrix**, denoted as $\mathbf{f}$.

*The mapping matrix $\mathbf{f}$ is a 2D matrix with dimensions: **[number of image pixels] × [number of source pixels]**, where:*

- A matrix entry of **0** means that the corresponding image pixel **does not** map to that source pixel.
- A matrix entry of **1** means that the image pixel **does** map to that source pixel.

Mapping Matrix

If $f[1,1] = 0$, the first image pixel **does not** map to the first source pixel.

If $f[1,1] = 1$, the first image pixel **does** map to the first source pixel.

Since each image pixel maps to only one source pixel, most entries in the mapping matrix are zero.

It is therefore very sparse.

Mapping Matrix via For Loop

JAX does not support code written like this (which originally used **numba**).

```
@numba_util.jit()
def mapping_matrix_from(
    source_index_for_image_index: np.ndarray,
    source_pixels: int,
    image_pixels: int,
) -> np.ndarray:

    mapping_matrix = np.zeros((image_pixels, source_pixels))

    for image_index in range(image_pixels):

        for source_index in range(source_index_for_image_index[image_index]):

            mapping_matrix[image_index][source_index] += 1

    return mapping_matrix
```

Mapping Matrix via For Loop

JAX does not support code written like this (which originally used **numba**).

Not allowed to use in place memory manipulation.

```
@numba_util.jit()
def mapping_matrix_from(
    source_index_for_image_index: np.ndarray,
    source_pixels: int,
    image_pixels: int,
) -> np.ndarray:

    mapping_matrix = np.zeros((image_pixels, source_pixels))

    for image_index in range(image_pixels):

        for source_index in range(source_index_for_image_index[image_index]):

            mapping_matrix[image_index][source_index] += 1

    return mapping_matrix
```

Mapping Matrix via For Loop

If you can use
vectorized
numpy (e.g.
broadcasting)
then adapting
code like this to
JAX is no
problem.

```
import jax
import jax.numpy as jnp

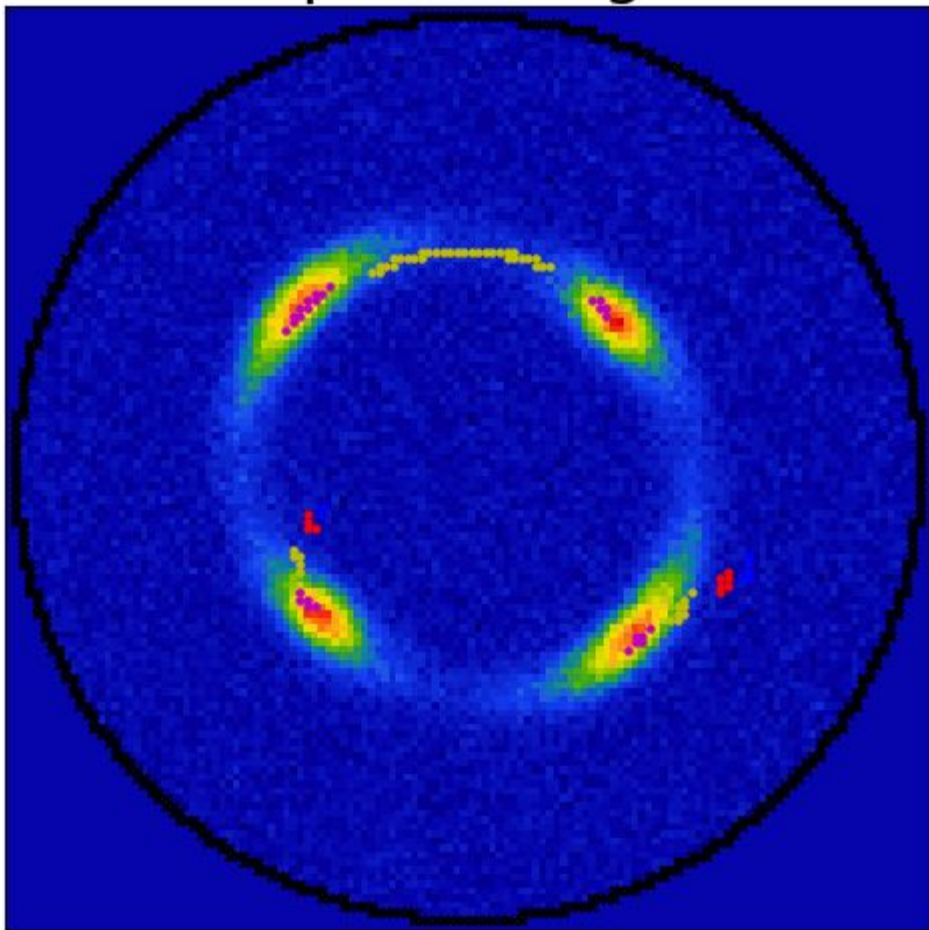
@jax.jit
def mapping_matrix_from(
    source_index_for_image_index: jnp.ndarray,
    source_pixels: int,
    image_pixels: int,
) -> jnp.ndarray:
    # shape: (image_pixels,)
    k = source_index_for_image_index[:image_pixels]

    # shape: (source_pixels,)
    cols = jnp.arange(source_pixels)

    # shape: (image_pixels, source_pixels)
    # entry (i, j) is 1 iff j < k[i], else 0
    return (cols[None, :] < k[:, None]).astype(jnp.float32)
```

Example 3: Open Source Libraries For Hard Tasks

Example Strong Lens



Source Reconstruction

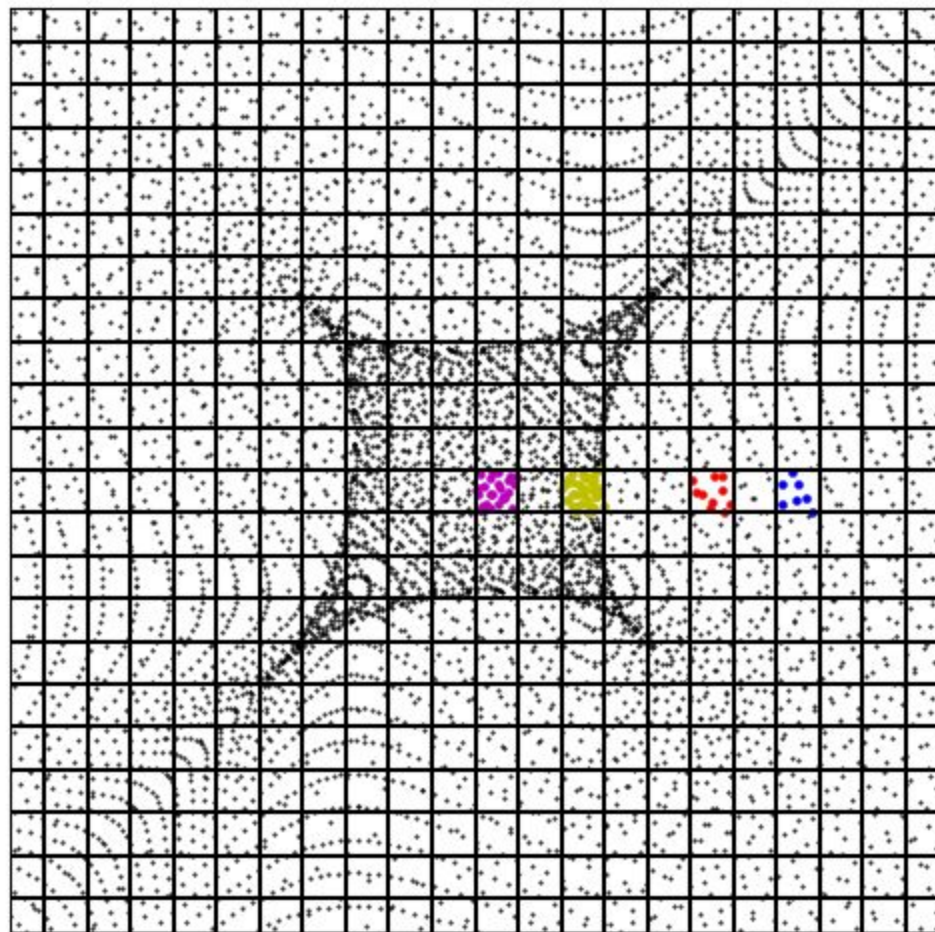
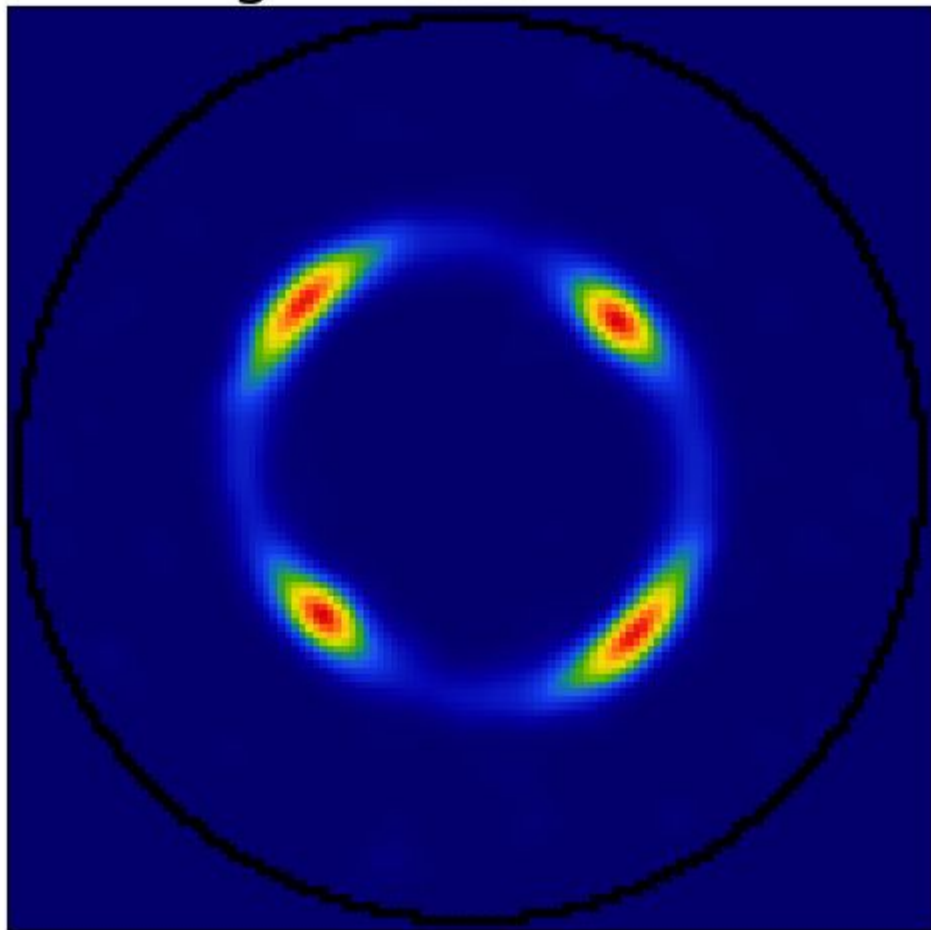
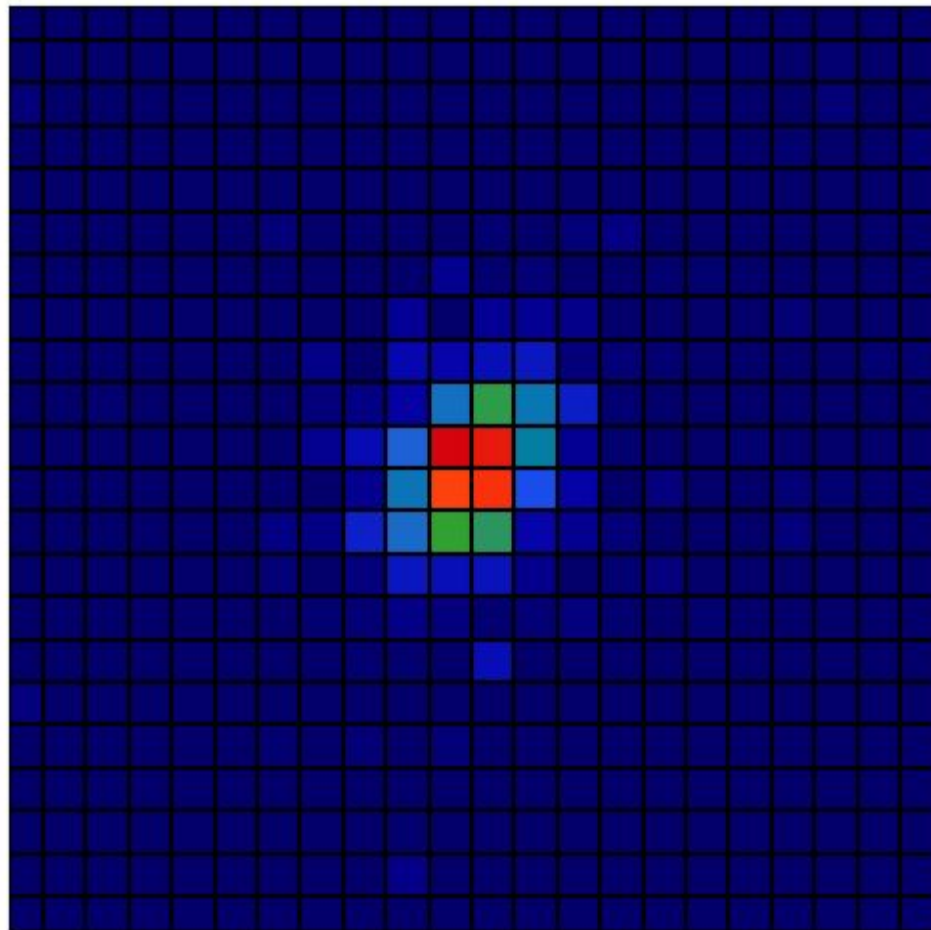


Image Reconstruction



Source Reconstruction



Linear Source Reconstruction

Use mapping matrix to set problem up as a large set of simultaneous linear equations, e.g:

$$33x + 16y + 72z = -359$$

$$-24x - 10y - 57z = 281$$

$$-8x - 4y - 17z = 85$$

Algebraically, can solve for x, y, z.

(number of variables to solve for = number of source pixels).

Linear Matrix Inversion

Setup as a matrix inversion:

$$A = \begin{pmatrix} 33 & 16 & 72 \\ -24 & -10 & -57 \\ -8 & -4 & -17 \end{pmatrix} \quad \text{and} \quad b = \begin{pmatrix} -359 \\ 281 \\ 85 \end{pmatrix}.$$

Linear Matrix Inversion

In Warren & Dye 2003, this is equivalent to:

$$\mathbf{S} = \mathbf{F}^{-1} \mathbf{D}$$

$\mathbf{F}_{ik} = \sum_{j=1}^J f_{ij} f_{kj} / \sigma_j^2$: Covariance between image and source pixels.

$\mathbf{D}_i = \sum_{j=1}^J f_{ij} d_j / \sigma_j^2$: Maps data to source pixels.

Linearly reconstruct the source that best fits the noise-weighted image data.

Linear Matrix Inversion

If we allow for **positive** and **negative** flux values of a source galaxy reconstruction, we would simply use:

```
reconstruction = np.linalg.solve(F, D)
```

JAX-ifying this is trivial, **because it mimics the required function in the numpy linalg library:**

```
reconstruction = jnp.linalg.solve(F, D)
```

Linear Matrix Inversion

However, scientifically we found that **requiring the solution to be positive-only** is necessary for the best science.

Pre-JAX, we used an open-source “**fast non negative least square (fnnls)**” solver we could on GitHub to do this efficiently:

<https://github.com/jvendrow/fnnls>

A slower version of the method **exists in scipy**:

<https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.optimize.nnls.html>

This has the potential to be a JAX brick wall, as the scipy method is not available in jax-scipy and converting the GitHub code to JAX would be very difficult.

Linear Matrix Inversion

On this occasion, the rich JAX open source ecosystem saved me!

- Found an open source package for **quadratic programming** with JAX called qpax.
- Adapted code to perform a fnnl for my problem.
- Fast, efficient and portable.

Did require me to read up on Quadratically programming, become more knowledgeable about linear algebra.

Was not a quick solution and has a high expertise requirement.

qpax

Differentiable QP solver in [JAX](#).

arXiv 2207.00669

This package can be used for solving convex quadratic programs of the following form:

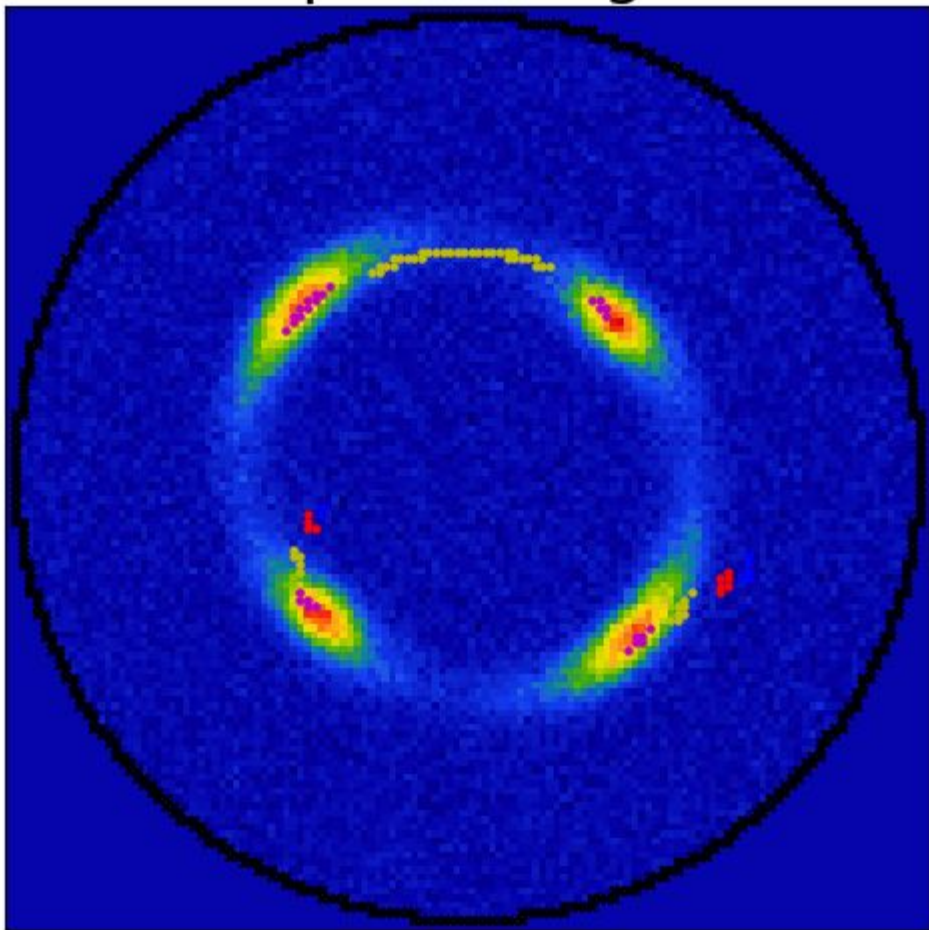
$$\begin{aligned} & \underset{x}{\text{minimize}} && \frac{1}{2}x^T Q x + q^T x \\ & \text{subject to} && Ax = b, \\ & && Gx \leq h \end{aligned}$$

where $Q \succeq 0$. This solver can be combined with JAX's `jit` and `vmap` functionality, as well as differentiated with reverse-mode `grad`.

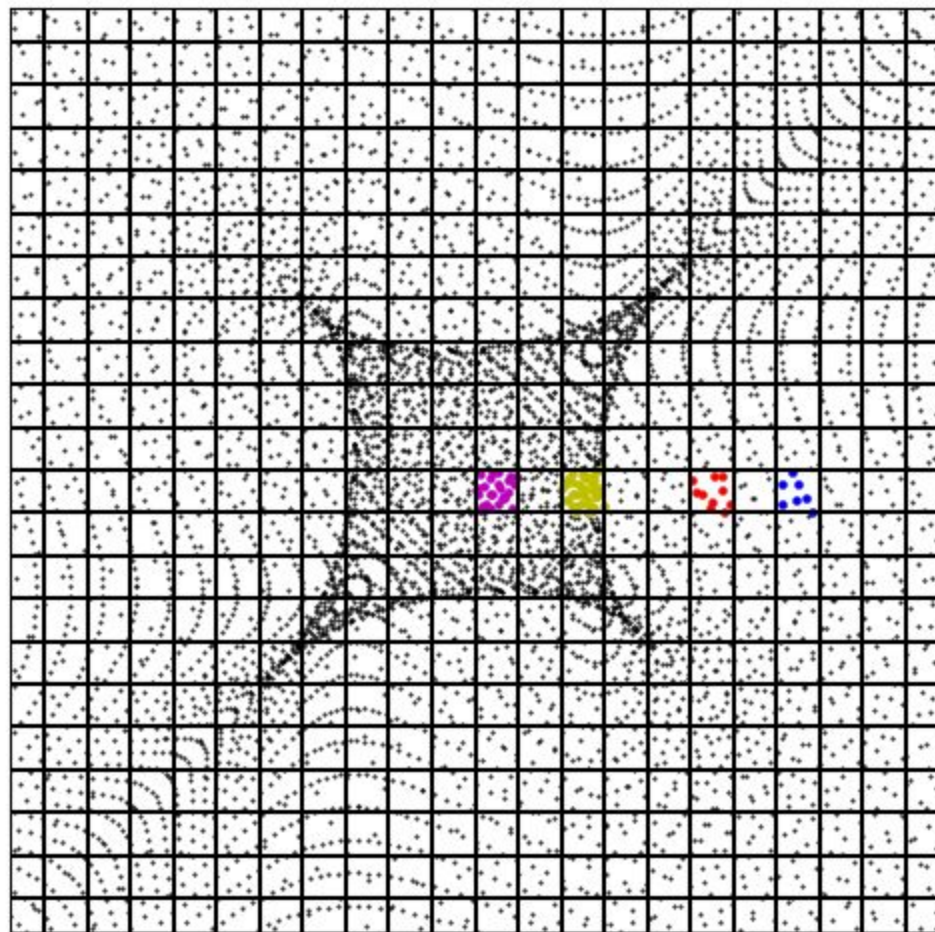
The QP is solved with a primal-dual interior point algorithm detailed in [cvxgen](#), with the solution to the linear systems computed with reduction techniques from [cvxopt](#). At an approximate primal-dual solution, the the primal variable x is differentiated with respect to the problem parameters using the implicit function theorem as shown in [optnet](#), and their pytorch-based qp solver [gpth](#).

Example 4: Brick Walls

Example Strong Lens

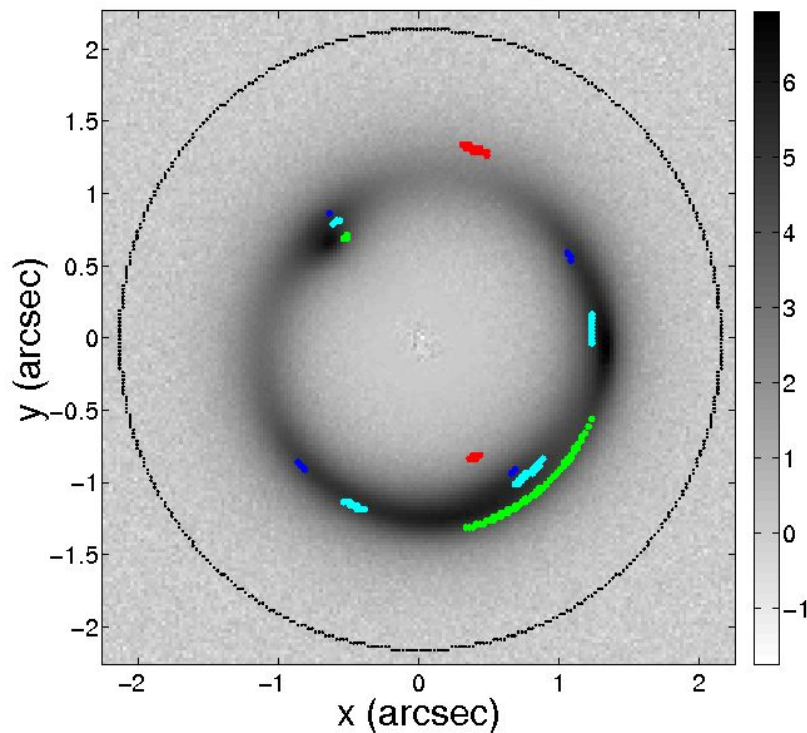


Source Reconstruction

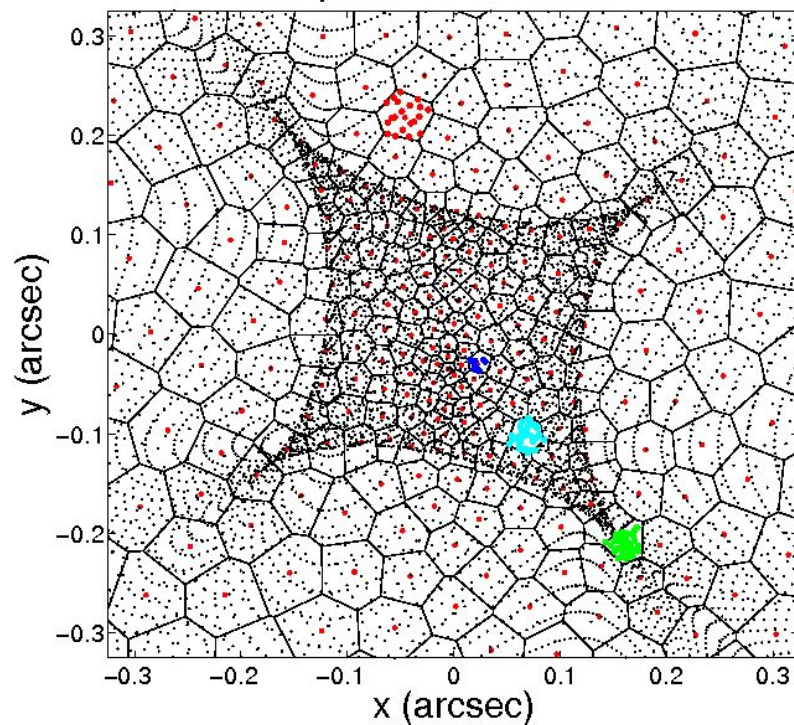


Using a Voronoi Mesh

Simulated Image



Adaptive Pixelization



Scipy Spatial: Voronoi

There is no native JAX support for **scipy.spatial** or Voronoi.

There is no open source JAX library which does a Voronoi mesh.

GPU Voronoi implementations that are available are discrete/approximate (grid-based distance transforms, jump-flooding, etc.), which can be fine for rendering or coarse assignment but aren't a robust replacement for exact computational geometry.

```
vor = scipy.spatial.Voronoi(  
    self.mesh_grid_xy,  
    qhull_options="Qbb Qc Qx Qm",  
)  
  
vertices = vor.vertices  
point_region = vor.point_region  
regions = vor.regions
```

Scipy Spatial: Voronoi

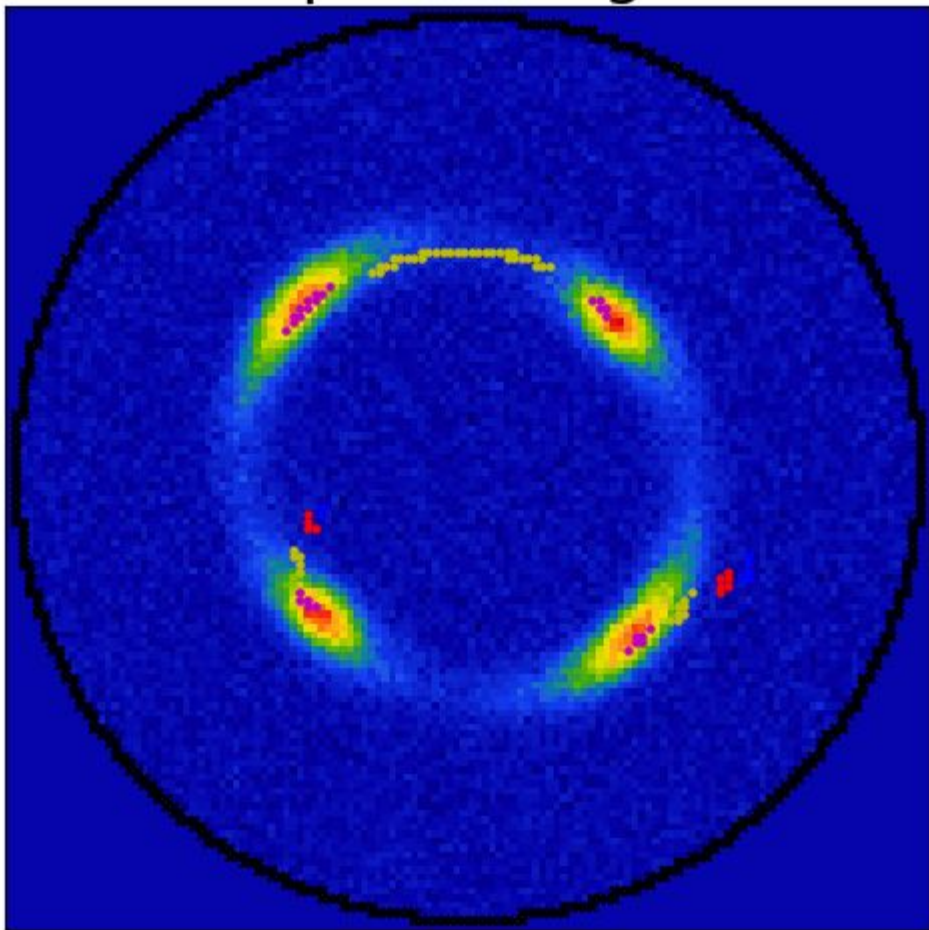
Exact Voronoi/Delaunay is built on adaptive, branchy, topology-changing algorithms with robust geometric predicates (orientation/in-circle tests) and lots of irregular control flow / dynamic data structures.

JAX (and GPUs) hate this.

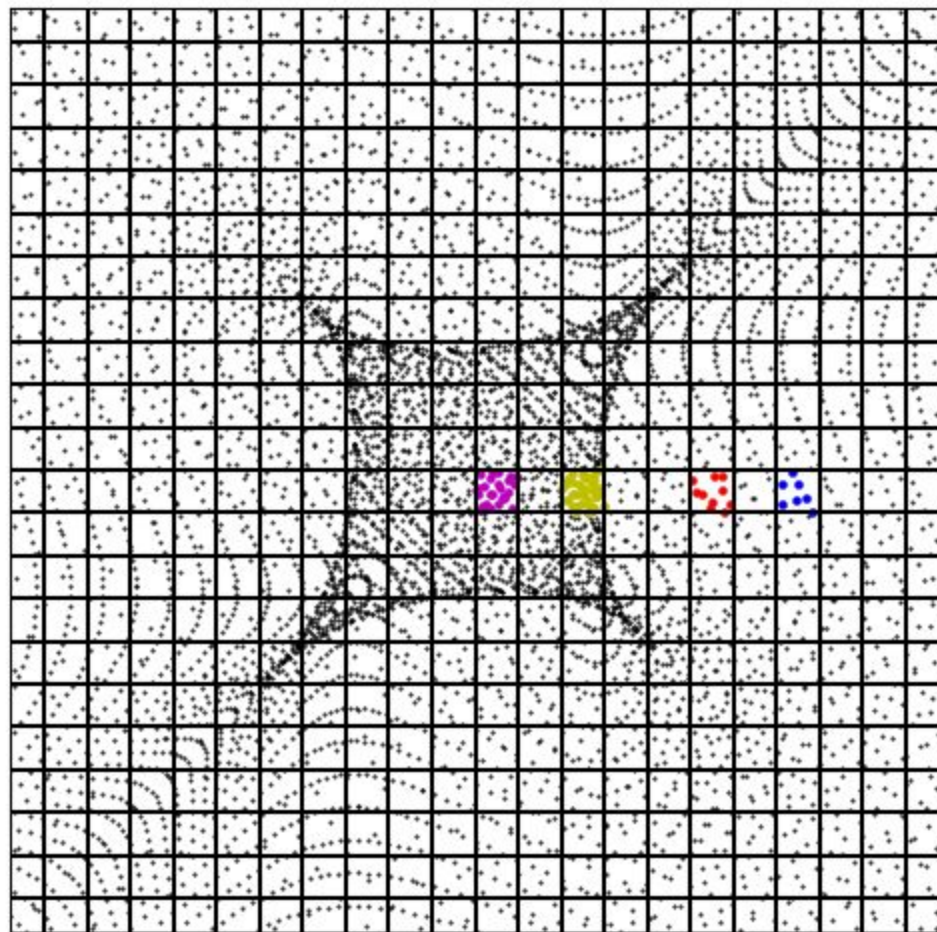
This is my JAX (and GPU) brick wall!

```
vor = scipy.spatial.Voronoi(  
    self.mesh_grid_xy,  
    qhull_options="Qbb Qc Qx Qm",  
)  
  
vertices = vor.vertices  
point_region = vor.point_region  
regions = vor.regions
```

Example Strong Lens



Source Reconstruction



Solution 2: Run on CPU

Used a JAX `pure_callback` to compute Voronoi/Delaunay meshes safely on the CPU

Geometry step runs outside JIT; avoids GPU-incompatible, topology-changing spatial algorithms.

Requires moving data between GPU VRAM and CPU memory for geometry calculations

Overhead negligible for my problem; JAX still accelerates the dominant linear algebra and inference steps

```
def jax_delaunay(points, query_points, use_voronoi_areas, areas_factor=0.5):
    import jax
    import jax.numpy as jnp

    N = points.shape[0]
    Q = query_points.shape[0]
    max_simplices = 2 * N

    points_shape = jax.ShapeDtypeStruct((N, 2), points.dtype)
    simplices_padded_shape = jax.ShapeDtypeStruct((max_simplices, 3), jnp.int32)
    mappings_shape = jax.ShapeDtypeStruct((Q, 3), jnp.int32)
    split_points_shape = jax.ShapeDtypeStruct((N * 4, 2), points.dtype)
    splitted_mappings_shape = jax.ShapeDtypeStruct((N * 4, 3), jnp.int32)

    return jax.pure_callback(
        lambda points, qpts: scipy_delaunay(
            np.asarray(points), np.asarray(qpts), use_voronoi_areas, areas_factor
        ),
        (
            points_shape,
            simplices_padded_shape,
            mappings_shape,
            split_points_shape,
            splitted_mappings_shape,
        ),
        points,
        query_points,
    )
```

Other JAX Remarks

Other JAX Remarks

Static array shapes are crucial in JAX: many algorithms must be reformulated to avoid dynamic sizes and control flow. For my problem, the mapping matrix being static is pivotal (e.g. number of image and source pixels is fixed)!

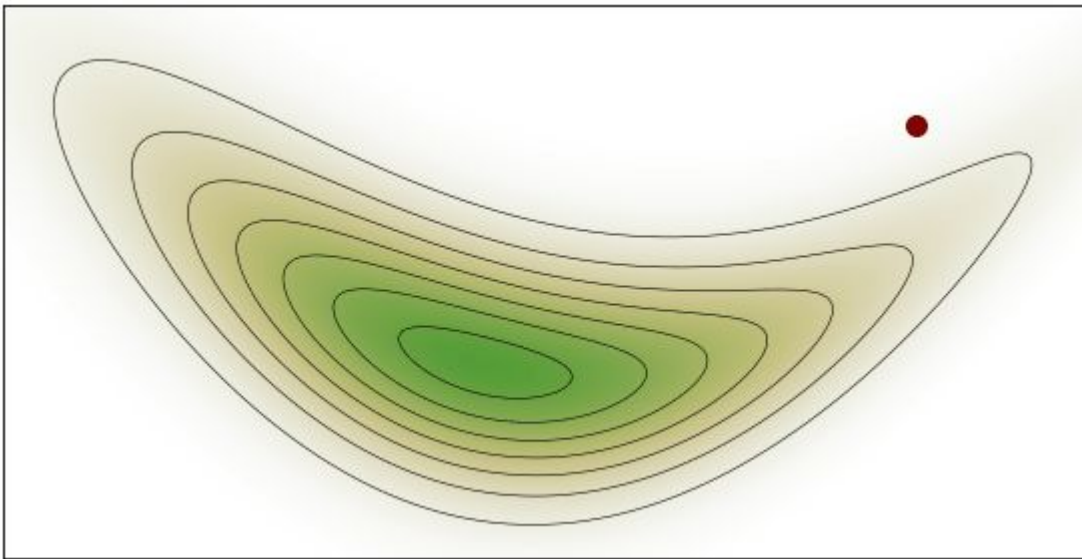
Exploiting sparsity is challenging: JAX has minimal support for sparse calculations due to its machine learning history, and exploiting sparsity is hard because memory must be ordered and static. If your existing CPU code massively exploits sparsity, significant speedup via JAX may require fundamental algorithmic redesigns.

JAX is the GPU library for doing Science: GPU libraries (E.g. Tensorflow, Pyro) are built for machine learning / deep learning, not custom scientific programming. **JAX changes this!**

Other JAX Remarks

Automatic differentiation and native gradients come almost for free in JAX, simplifying inference, optimization, and sensitivity analysis without bespoke derivative code.

Hamiltonian Monte Carlo
MCMC now possible in PyAutoLens, I thought this was impossible 3 years ago!



**Was JAX Development
Worth it?**

Is JAX Development Worth it?

Speed: Modeling which previously took 5 hours now takes 15 minutes, or which previously took 24 hours now takes < 1 hour, with more speed ups to come. **100x speed up is feasible for suitable projects, scientifically undoubtedly worth it!**

Portability: I released PyAutoLens-JAX publicly in October and all users who have upgraded had little to no issue whatsoever getting it running on GPU. **JAX is a truly portable and widely used open source GPU library!**

Scientific Analysis: Auto Differentiation and gradients unlock a host of extra benefits which I have not touched on, **makes your code way more power!**

Agentic AI

ChatGPT is amazing

LLM's are amazing for converting code where **you know the output for a given input**.

CPU to GPU -> compelling use case.

Step 1: write a *pure NumPy* helper

This must:

- take a NumPy array
- return a NumPy array
- do *exactly* what your current method does

python

 Copy code

```
def _pynufft_forward_numpy(self, image_np: np.ndarray) -> np.ndarray:
    """
    NumPy-only forward NUFFT. Runs on host.
    """
    warnings.filterwarnings("ignore")

    # Flip vertically (PyNUFFT internal convention)
    image_np = image_np[::-1, :]

    # PyNUFFT forward
    vis = self.forward(image_np)

    return vis
```

 This function **must not touch JAX**.

Step 2: wrap it with `jax.pure_callback`

Now we expose it to JAX.

python

 Copy code

```
import jax
import jax.numpy as jnp
from jax import ShapeDtypeStruct
```

Using ChatGPT

RSE worked on project for 3 months aiming to speed up interferometry analysis in PyAutoLens with JAX.

The end conclusion was that given the current implementation it was not possible due to how JAX has to scatter values in memory.



Using ChatGPT

A few months later I had a 40 minute conversation with ChatGPT about the problem (around November 2025).

A few days later I had **code that produced a x100+ speed up for JAX on GPU.**

ChatGPT basically had knowledge and expertise of linear algebra neither i or the RSE had.



Sat, Sep 5 at 2:06 PM

 **GitHub** on autolens_profiling we have done lots of work investigator the break down run of the likelihood functions using a pixelized mesh using jax. In particular the delaunay mesh on imaging data
(https://github.com/PyAutoLabs/autolens_profiling/blob/main/scripts%2Fimaging%2Flikelihood_breakdown%2Fdelaunay.py).

We have generally concluded the bottle necks are that for JAX on an A100 GPU the nnls method is hard if not impossible to speed up and the delaunay mesh is a bottleneck that can't be sped up.

Show more ▾



1. The strongest immediate candidate: early termination of Delaunay point location.

Your current implementation already keeps only Qhull triangulation on CPU; point location, barycentric weights, areas and split points run in JAX. So the earlier proposal to move `find_simplex` into JAX has already been implemented.

However, `pix_indexes_delaunay_walk_from` currently:

- Finds a nearest vertex using all query–vertex distances.
- Processes queries in chunks of 1,024.
- Runs **all 128 visibility-walk iterations**, even when every point in a chunk is already resolved.
- Repeats this process for the regularization split points.

The NumPy version exits early; the JAX version does not. This is a specific remaining opportunity in the current implementation ↗.

My first experiment would replace the fixed walk with a bounded `lax.while_loop`, stopping when all queries are resolved. Keep the existing cap and fallback semantics.

Frontier Models (Fable / Astra)

- 1) **Expertise:** They get most things right now, and know more than me on the mathematics and foundations on which my code is based (linear algebra, spatial geometry, Fourier Transforms, etc).
- 2) **Time is no longer a constraint:** Finding the faster solution often was constrained by time, you couldnt “trial and error” everything to see what would work, this is no longer the case.
- 3) **Its now okay to write complex optimized code.**

